Mechanised Functional Verification of Mixed Choice Multiparty Message Passing

Jonas Kastberg Hinrichsen
Aalborg University

Iwan Quémerais
ENS-Lyon

Lars Birkedal
Aarhus University

Accepted at OOPSLA'26 (Camera Ready Deadline 27. February)

10. February, Dagstuhl Seminar 26071

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

We lift the properties to functional correctness as:

- **Uniqueness:** The leader can proceed with elevated permissions
- **Agreement:** Participant interaction can depend on known leader

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

We lift the properties to functional correctness as:

- **Uniqueness:** The leader can proceed with elevated permissions
- **Agreement:** Participant interaction can depend on known leader

We consider process-level implementation details

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

We lift the properties to functional correctness as:

- **Uniqueness:** The leader can proceed with elevated permissions
- **Agreement:** Participant interaction can depend on known leader

We consider process-level implementation details

Goal: Mechanised functional correctness proofs for leader election implementations alongside other programming paradigms

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

We lift the properties to functional correctness as:

- **Uniqueness:** The leader can proceed with elevated permissions
- **Agreement:** Participant interaction can depend on known leader

We consider process-level implementation details

Goal: Mechanised functional correctness proofs for leader election implementations alongside other programming paradigms

Key idea: Mixed choice session types meets separation logic

Leader Election

Given a set of actors, leader election is an algorithm that satisfies:

- **Uniqueness:** There is exactly one actor that considers itself as leader <——
- **Agreement:** All other actors know who the leader is
- **Termination:** The algorithm finishes in finite time

We lift the properties to functional correctness as:

- **Uniqueness:** The leader can proceed with elevated permissions <——
- **Agreement:** Participant interaction can depend on known leader

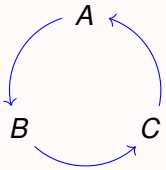
We consider process-level implementation details

Goal: Mechanised functional correctness proofs for leader election implementations alongside other programming paradigms

Key idea: Mixed choice session types meets separation logic

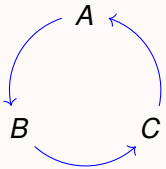
Mixed Choice in π-Calculus

Mixed Choice: simultaneously trying to send to- and receive from multiple parties



Mixed Choice in π-Calculus

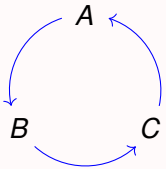
Mixed Choice: simultaneously trying to send to- and receive from multiple parties



$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid\ e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \\
\mid\mid\ e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

Mixed Choice in π-Calculus

Mixed Choice: simultaneously trying to send to- and receive from multiple parties

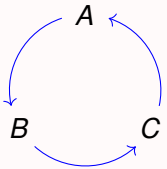


$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Mixed Choice in π-Calculus

Mixed Choice: simultaneously trying to send to- and receive from multiple parties



$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

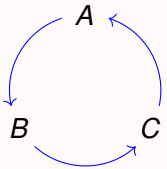
$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Suitable for describing and verifying safety of leader election

- Mixed choice allows "breaking the symmetry" of processes [Palamidessi'03]
- Mixed choice session types allows scalable verification [Peters & Yoshida'24]

Mixed Choice in π-Calculus

Mixed Choice: simultaneously trying to send to- and receive from multiple parties



$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Suitable for describing and verifying safety of leader election

- Mixed choice allows "breaking the symmetry" of processes [Palamidessi'03]
- Mixed choice session types allows scalable verification [Peters & Yoshida'24]

Problem: No work on functional correctness nor mechanisation of mixed choice

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Add or implement GV-style endpoints-as-terms message passing primitives

- $\mathbf{new_chan}\ (), c.\mathbf{send}(v), c.\mathbf{recv}()$

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Add or implement GV-style endpoints-as-terms message passing primitives

- $\mathbf{new_chan}\ (), c.\mathbf{send}(v), c.\mathbf{recv}()$

Prove verification rules for message passing primitives

- $\mathsf{wp}\ \mathbf{new_chan}\ ()\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{send}(v)\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{recv}()\ \{\ldots\}$

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Add or implement GV-style endpoints-as-terms message passing primitives

- $\mathbf{new_chan}\ (), c.\mathbf{send}(v), c.\mathbf{recv}()$

Prove verification rules for message passing primitives

- $\mathsf{wp}\ \mathbf{new_chan}\ ()\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{send}(v)\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{recv}()\ \{\ldots\}$

Prove WP's of programs using logic

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Add or implement GV-style endpoints-as-terms message passing primitives

- $\mathbf{new_chan}\ (), c.\mathbf{send}(v), c.\mathbf{recv}()$

Prove verification rules for message passing primitives

- $\mathsf{wp}\ \mathbf{new_chan}\ ()\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{send}(v)\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{recv}()\ \{\ldots\}$

Prove WP's of programs using logic
Previously: deadlock freedom, distributed systems, and multiparty

Functional Correctness of Message Passing

Actris: Separation logic for message passing in Iris mechanised in Rocq

- $\vdash \mathsf{wp}\ e\ \{v.\ \mathsf{True}\} \Rightarrow \mathtt{safe}(e)$

Start with an OCaml-like language with shared memory and concurrency

- $e := () \mid z \mid \ell \mid e; e \mid \mathbf{let}\ x := e\ \mathbf{in}\ e \mid \mathbf{ref}\ e \mid \mathbf{free}\ e \mid \mathbf{fork}\ \{e\} \mid \ldots$

Add or implement GV-style endpoints-as-terms message passing primitives

- $\mathbf{new_chan}\ (), c.\mathbf{send}(v), c.\mathbf{recv}()$

Prove verification rules for message passing primitives

- $\mathsf{wp}\ \mathbf{new_chan}\ ()\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{send}(v)\ \{\ldots\}$ $\quad$ $\mathsf{wp}\ c.\mathbf{recv}()\ \{\ldots\}$

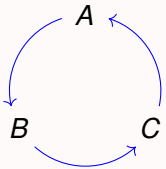
Prove WP's of programs using logic
Previously: deadlock freedom, distributed systems, and multiparty

Problem: Not much work on mixed choice in non π-calculus settings

- "Mixed choice is a really difficult mechanism to implement" [Palamidessi'03]

Mixed Choice in Non π-Calculus

Per-thread semantics: Threads take turns (arbitrarily) reducing individually

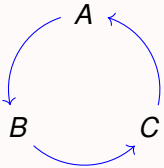


$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow^* \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Mixed Choice in Non π-Calculus

Per-thread semantics: Threads take turns (arbitrarily) reducing individually



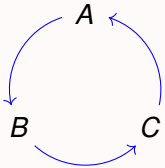
$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow^* \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Non-collaborative concurrency: Threads cannot impact scheduling of others

Mixed Choice in Non π-Calculus

Per-thread semantics: Threads take turns (arbitrarily) reducing individually



$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow^* \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

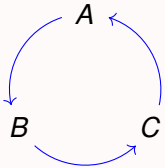
$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Non-collaborative concurrency: Threads cannot impact scheduling of others

- Threads instead take turns trying to handshake until success

Mixed Choice in Non π-Calculus

Per-thread semantics: Threads take turns (arbitrarily) reducing individually



$$
\begin{aligned}
e_A &:= ![B]\langle v_1\rangle.e_{A1} + ?[C](x_3).e_{A2} \\
\mid\mid e_B &:= ![C]\langle v_2\rangle.e_{B1} + ?[A](x_1).e_{B2} \qquad \Rightarrow^* \\
\mid\mid e_C &:= ![A]\langle v_3\rangle.e_{C1} + ?[B](x_2).e_{C2}
\end{aligned}
$$

$$e_{A1} \mid\mid e_{B2}[v_1/x_1] \mid\mid e_C \qquad e_A \mid\mid e_{B1} \mid\mid e_{C2}[v_2/x_2] \qquad e_{A2}[v_3/x_3] \mid\mid e_B \mid\mid e_{C1}$$

Non-collaborative concurrency: Threads cannot impact scheduling of others

- Threads instead take turns trying to handshake until success

Risk of repeatedly missing each other

- Although we have almost-sure termination under uniform scheduling

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory

- With safe synchronisation between participants

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory

- With safe synchronisation between participants

Mixed Choice Multiparty Dependent Separation Protocols (MCMDSPs)

- Rich protocols for describing mixed choice multiparty message passing

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory

- With safe synchronisation between participants

Mixed Choice Multiparty Dependent Separation Protocols (MCMDSPs)

- Rich protocols for describing mixed choice multiparty message passing

Mixtris separation logic

- Bottom-up protocol consistency proven in separation logic
- Logic for verifying mixed choice multiparty communication via MCMDSPs
- Verification rules proven against channel implementation

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory

- With safe synchronisation between participants

Mixed Choice Multiparty Dependent Separation Protocols (MCMDSPs)

- Rich protocols for describing mixed choice multiparty message passing

Mixtris separation logic

- Bottom-up protocol consistency proven in separation logic
- Logic for verifying mixed choice multiparty communication via MCMDSPs
- Verification rules proven against channel implementation

Verification suite of mixed choice multiparty programs

- Threeway election
- Chang and Roberts's ring leader election algorithm

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory

- With safe synchronisation between participants

Mixed Choice Multiparty Dependent Separation Protocols (MCMDSPs)

- Rich protocols for describing mixed choice multiparty message passing

Mixtris separation logic

- Bottom-up protocol consistency proven in separation logic
- Logic for verifying mixed choice multiparty communication via MCMDSPs
- Verification rules proven against channel implementation

Verification suite of mixed choice multiparty programs

- Threeway election
- Chang and Roberts's ring leader election algorithm

Full mechanisation in Rocq

- With automation support

Contributions

Implementation of Mixed Choice Multiparty Channels in shared memory <——

- With safe synchronisation between participants

Mixed Choice Multiparty Dependent Separation Protocols (MCMDSPs) <——

- Rich protocols for describing mixed choice multiparty message passing

Mixtris separation logic <——

- Bottom-up protocol consistency proven in separation logic
- Logic for verifying mixed choice multiparty communication via MCMDSPs
- Verification rules proven against channel implementation

Verification suite of mixed choice multiparty programs

- Threeway election <——
- Chang and Roberts's ring leader election algorithm

Full mechanisation in Rocq

- With automation support

Implementation of Mixed Choice

Mixed Choice Channel Endpoint Abstraction

Mixed choice channel endpoint API:

$\texttt{new_chan}(n)$	Creates a mixed choice multiparty channel with n parties, returning a tuple $(c_0, ..., c_{(n-1)})$ of endpoints
$c_i[j].\texttt{try_send}(v)$	Momentarily tries to send value v via endpoint c_i to party j; returns boolean signifying success
$c_i[j].\texttt{try_recv}()$	Momentarily tries to receive a value via endpoint c_i to party j; returns option of result

Mixed Choice Channel Endpoint Abstraction

Mixed choice channel endpoint API:

$\texttt{new_chan}(n)$	Creates a mixed choice multiparty channel with n parties, returning a tuple $(c_0, \ldots, c_{(n-1)})$ of endpoints
$c_i[j].\texttt{try_send}(v)$	Momentarily tries to send value v via endpoint c_i to party j; returns boolean signifying success
$c_i[j].\texttt{try_recv}()$	Momentarily tries to receive a value via endpoint c_i to party j; returns option of result

Encoding of simultaneous binary choice:

$$
\begin{aligned}
\texttt{send_recv}\ c\ i\ j\ v \triangleq\ & \textbf{if}\ c[i].\texttt{try_send}(v)\ \textbf{then}\ \textbf{none} \\
& \textbf{else}\ \textbf{match}\ c[j].\texttt{try_recv}()\ \textbf{with} \\
& \quad \mid \textbf{some}\ x \Rightarrow \textbf{some}\ x \\
& \quad \mid \textbf{none} \quad \Rightarrow \texttt{send_recv}\ c\ i\ j\ v \\
& \quad \textbf{end}
\end{aligned}
$$

Implementation of Mixed Choice Channel

Matrix of **synchronisation cells**, where i,j is used to send from i to j and vice versa

Implementation of Mixed Choice Channel

Matrix of **synchronisation cells**, where i,j is used to send from i to j and vice versa:

	0	**...**	**i**	**...**	**j**	**...**	**n-1**
0	-	...	$0 \to i$	...	$0 \to j$	...	$0 \to n\text{-}1$
...	...	...	...	...	...	...	...
i	$i \to 0$	...	-	...	$i \to j$	...	$i \to n\text{-}1$
...	...	...	...	...	...	...	...
j	$j \to 0$	...	$j \to i$	...	-	...	$j \to n\text{-}1$
...	...	...	...	...	...	...	...
n-1	$n\text{-}1 \to 0$	...	$n\text{-}1 \to i$	...	$n\text{-}1 \to j$	...	-

Implementation of Mixed Choice Channel

Matrix of **synchronisation cells**, where i,j is used to send from i to j and vice versa:

	0	**...**	**i**	**...**	**j**	**...**	**n-1**
0	-	...	$0 \to i$	...	$0 \to j$	...	$0 \to n\text{-}1$
...	...	...	...	...	...	...	...
i	$i \to 0$	...	-	...	$i \to j$	...	$i \to n\text{-}1$
...	...	...	...	...	...	...	...
j	$j \to 0$	...	$j \to i$	...	-	...	$j \to n\text{-}1$
...	...	...	...	...	...	...	...
n-1	$n\text{-}1 \to 0$	...	$n\text{-}1 \to i$	...	$n\text{-}1 \to j$	...	-

Channel endpoint c_i: Tuple of shared reference to matrix and own id i

Implementation of Mixed Choice Channel

Matrix of **synchronisation cells**, where i,j is used to send from i to j and vice versa:

	0	**...**	**i**	**...**	**j**	**...**	**n-1**
0	-	...	$0 \to i$	...	$0 \to j$	...	$0 \to n\text{-}1$
...	...	...	...	...	...	...	...
i	$i \to 0$	...	-	...	$i \to j$	...	$i \to n\text{-}1$
...	...	...	...	...	...	...	...
j	$j \to 0$	...	$j \to i$	...	-	...	$j \to n\text{-}1$
...	...	...	...	...	...	...	...
n-1	$n\text{-}1 \to 0$	...	$n\text{-}1 \to i$	...	$n\text{-}1 \to j$	...	-

Channel endpoint c_i: Tuple of shared reference to matrix and own id i

- Send/Recv look up and use corresponding synchronisation cell in matrix

Implementation of Mixed Choice Channel

Matrix of **synchronisation cells**, where i,j is used to send from i to j and vice versa:

	0	**...**	**i**	**...**	**j**	**...**	**n-1**
0	-	...	$0 \to i$	...	$0 \to j$	...	$0 \to n\text{-}1$
...	...	...	...	...	...	...	...
i	$i \to 0$	...	-	...	$i \to j$	...	$i \to n\text{-}1$
...	...	...	...	...	...	...	...
j	$j \to 0$	...	$j \to i$	...	-	...	$j \to n\text{-}1$
...	...	...	...	...	...	...	...
n-1	$n\text{-}1 \to 0$	...	$n\text{-}1 \to i$	...	$n\text{-}1 \to j$	...	-

Channel endpoint c_i: Tuple of shared reference to matrix and own id i

- Send/Recv look up and use corresponding synchronisation cell in matrix
- Synchronisation cells ensure appropriate synchronisation guarantees

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where

Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

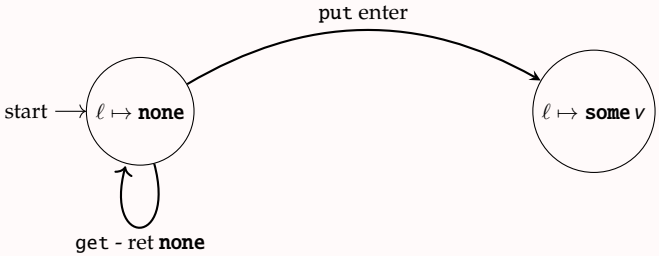
Synchronisation cells: Reference ℓ with `put` and `get` where

Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where

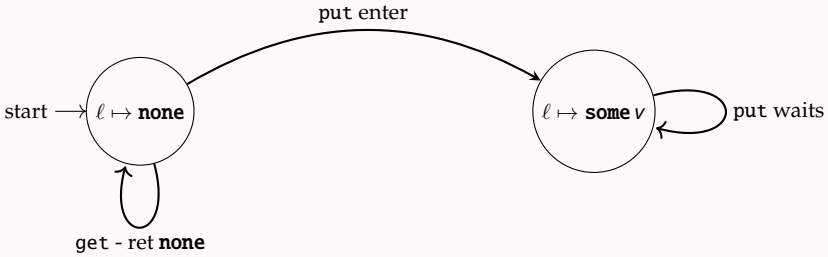


Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where

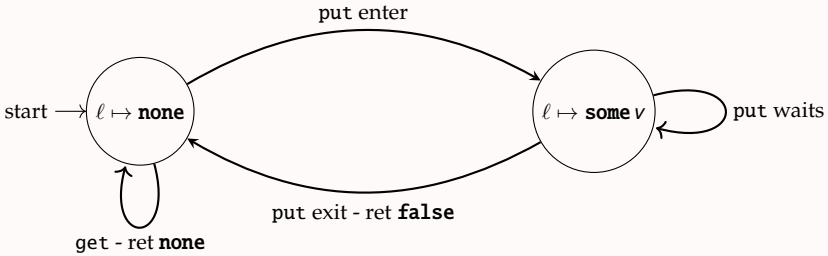


Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where

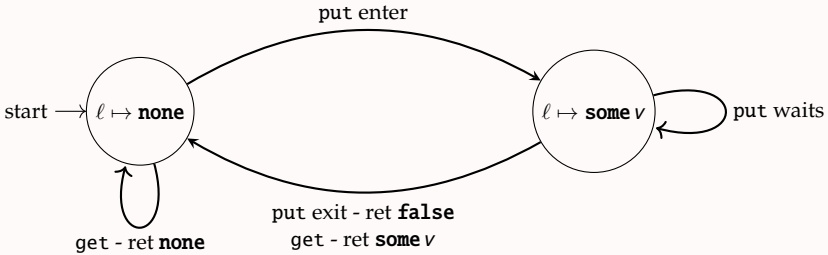


Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where

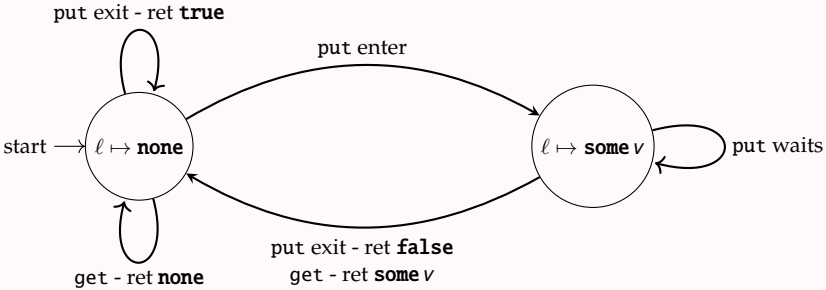


Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Synchronisation Cells

Synchronisation cells: Reference ℓ with `put` and `get` where



Synchronisation guarantees:

- Success when sender commits, receiver commits, sender observes
- Failure when sender commits, sender aborts, receiver tries to commit
- Guarantees synchronicity; sender/receiver both fail or succeed together

Implementation of Threeway Election

Example of encoded simultaneous binary choice:

$$
\begin{aligned}
\texttt{send_recv}\ c\ i\ j\ v \triangleq\ & \textbf{if}\ c[i].\textbf{try_send}(v)\ \textbf{then none} \\
& \textbf{else match}\ c[j].\textbf{try_recv}()\ \textbf{with} \\
& \quad \mid \textbf{some}\ x \Rightarrow \textbf{some}\ x \\
& \quad \mid \textbf{none} \quad \Rightarrow \texttt{send_recv}\ c\ i\ j\ v \\
& \quad \textbf{end}
\end{aligned}
$$

Implementation of threeway election: (where A:=0, B:=1, C:=2)

$$
\begin{aligned}
& \texttt{threeway_election_example} \triangleq \\
& \quad \textbf{let}\ \ell := \textbf{ref}\ 42\ \textbf{in} \\
& \quad \textbf{let}\ (c_A, c_B, c_C) := \textbf{new_chan}(3)\ \textbf{in} \\
& \quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \textbf{with none} \Rightarrow (); \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}; \\
& \quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \textbf{with none} \Rightarrow (); \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}; \\
& \quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \textbf{with none} \Rightarrow (); \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}
\end{aligned}
$$

Mixed Choice Multiparty Dependent Separation Protocols

Leader Uniqueness

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \textbf{let}\ \ell := \textbf{ref}\ 42\ \textbf{in} \\
&\quad \textbf{let}\ (c_A, c_B, c_C) := \textbf{new_chan}(3)\ \textbf{in} \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\,; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\,; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}
\end{aligned}
$$

Goal: Prove leader uniqueness (no double-free)

Leader Uniqueness

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \textbf{let}\ \ell := \textbf{ref}\ 42\ \textbf{in} \\
&\quad \textbf{let}\ (c_A, c_B, c_C) := \textbf{new_chan}(3)\ \textbf{in} \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\ ; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\ ; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}
\end{aligned}
$$

Goal: Prove leader uniqueness (no double-free)

Mixed choice multiparty session types:

$c_A : ![B].\,\mathbf{end} + ?[C].\,\mathbf{end}$ $\quad$ $c_B : ![C].\,\mathbf{end} + ?[A].\,\mathbf{end}$ $\quad$ $c_C : ![A].\,\mathbf{end} + ?[B].\,\mathbf{end}$

Leader Uniqueness

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \textbf{let}\ \ell := \textbf{ref}\ 42\ \textbf{in} \\
&\quad \textbf{let}\ (c_A, c_B, c_C) := \textbf{new_chan}(3)\ \textbf{in} \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\ ; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}\ ; \\
&\quad \textbf{fork}\ \{\textbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \textbf{with}\ \textbf{none} \Rightarrow ();\ \textbf{some}\ _ \Rightarrow \textbf{free}\ \ell\ \textbf{end}\}
\end{aligned}
$$

Goal: Prove leader uniqueness (no double-free)

Mixed choice multiparty session types:

$$c_A : ![B].\,\textbf{end} + ?[C].\,\textbf{end} \qquad c_B : ![C].\,\textbf{end} + ?[A].\,\textbf{end} \qquad c_C : ![A].\,\textbf{end} + ?[B].\,\textbf{end}$$

Problem: No way of associating elected leader with privilege to free

Mixed Choice Multiparty Dependent Separation Protocols

Enriching protocols with separation logic resources

- $!\,[B].\,\mathbf{end} + ?[C].\,\mathbf{end} \quad \rightarrow \quad !\,[B]\,.\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end}$

Mixed Choice Multiparty Dependent Separation Protocols

Enriching protocols with separation logic resources

- $\mathbf{!}[B].\,\mathbf{end} + \mathbf{?}[C].\,\mathbf{end} \quad \rightarrow \quad \mathbf{!}\,[B]\,.\,\mathbf{end} + \mathbf{?}[C]\,\{\ell \mapsto 42\}.\,\mathbf{end}$

Mixed choice multiparty dependent separation protocols (OBS: Simplified)

- $p ::= \mathbf{!}\,[i]\,\{P\}.\,p \mid \mathbf{?}[i]\,\{P\}.\,p \mid p + q \mid \mathbf{end}$

Mixed Choice Multiparty Dependent Separation Protocols

Enriching protocols with separation logic resources

- $![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \quad \rightarrow \quad ![B]\,.\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end}$

Mixed choice multiparty dependent separation protocols (OBS: Simplified)

- $p ::= ![i]\,\{P\}.\,p \mid ?[i]\,\{P\}.\,p \mid p + q \mid \mathbf{end}$

Sufficient for describing threeway leader process

$$\mathtt{match}\ \mathtt{send_recv}\ c_A\ B\ C\ ()\ \mathtt{with}\ \mathtt{none} \Rightarrow ();\ \mathtt{some}\ _ \Rightarrow \mathtt{free}\ \ell\ \mathtt{end}$$

Mixed Choice Multiparty Dependent Separation Protocols

Enriching protocols with separation logic resources

- $![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \quad \rightarrow \quad ![B].\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end}$

Mixed choice multiparty dependent separation protocols (OBS: Simplified)

- $p ::= ![i]\,\{P\}.\,p \mid ?[i]\,\{P\}.\,p \mid p + q \mid \mathbf{end}$

Sufficient for describing threeway leader process

$$\mathbf{match}\ \mathtt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}$$

Remaining challenges:

- How to guarantee consistent global communication?
- How to apply protocols to verify program?

The Mixtris Separation Logic

Protocol Consistency

Remaining challenge: How to guarantee consistent global communication?

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Protocol Consistency

Remaining challenge: How to guarantee consistent global communication?

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Prior work: Syntactic duality

$$
\begin{aligned}
c_A &: ![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \\
c_B &: ![C].\,\mathbf{end} + ?[A].\,\mathbf{end} \\
c_C &: ![A].\,\mathbf{end} + ?[B].\,\mathbf{end}
\end{aligned}
$$

Protocol Consistency

Remaining challenge: How to guarantee consistent global communication?

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Prior work: Syntactic duality

$$
\begin{aligned}
c_A &: ![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \\
c_B &: ![C].\,\mathbf{end} + ?[A].\,\mathbf{end} \\
c_C &: ![A].\,\mathbf{end} + ?[B].\,\mathbf{end}
\end{aligned}
$$

This work:

$$
\begin{aligned}
c_A &\rightarrowtail ![B].\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_B &\rightarrowtail ![C].\,\mathbf{end} + ?[A]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_C &\rightarrowtail ![A].\,\mathbf{end} + ?[B]\,\{\ell \mapsto 42\}.\,\mathbf{end}
\end{aligned}
$$

Protocol Consistency

Remaining challenge: How to guarantee consistent global communication?

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Prior work: Syntactic duality

$$
\begin{aligned}
c_A &: ![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \\
c_B &: ![C].\,\mathbf{end} + ?[A].\,\mathbf{end} \\
c_C &: ![A].\,\mathbf{end} + ?[B].\,\mathbf{end}
\end{aligned}
$$

This work: Contextual semantic duality

$$
\begin{aligned}
c_A &\rightarrowtail ![B].\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_B &\rightarrowtail ![C].\,\mathbf{end} + ?[A]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_C &\rightarrowtail ![A].\,\mathbf{end} + ?[B]\,\{\ell \mapsto 42\}.\,\mathbf{end}
\end{aligned}
$$

Protocol Consistency

Remaining challenge: How to guarantee consistent global communication?

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Prior work: Syntactic duality

$$
\begin{aligned}
c_A &: ![B].\,\mathbf{end} + ?[C].\,\mathbf{end} \\
c_B &: ![C].\,\mathbf{end} + ?[A].\,\mathbf{end} \\
c_C &: ![A].\,\mathbf{end} + ?[B].\,\mathbf{end}
\end{aligned}
$$

This work: Contextual semantic duality

$$
\begin{aligned}
c_A &\rightarrowtail\, ![B]\,.\,\mathbf{end} + ?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_B &\rightarrowtail\, ![C]\,.\,\mathbf{end} + ?[A]\,\{\ell \mapsto 42\}.\,\mathbf{end} \\
c_C &\rightarrowtail\, ![A]\,.\,\mathbf{end} + ?[B]\,\{\ell \mapsto 42\}.\,\mathbf{end}
\end{aligned}
$$

Key Idea: Define and prove consistency via separation logic!

Protocol Consistency

We define a set of protocols $p_0, \ldots, p_k$ to be CONSISTENT $(p_0, \ldots, p_k)$ whenever, for any synchronised pair i, j, given the resources of i, and any prior resources

1. Provide the resources of j
2. Prove protocol consistency where i and j are updated to their respective tails

Repeat until no more synchronised pairs exist.

Protocol Consistency

We define a set of protocols $p_0, \ldots, p_k$ to be $\textsc{consistent}\ (p_0, \ldots, p_k)$ whenever, for any synchronised pair i, j, given the resources of i, and any prior resources

1. Provide the resources of j
2. Prove protocol consistency where i and j are updated to their respective tails

Repeat until no more synchronised pairs exist.

Given $\ell \mapsto 42$, show consistency of

$$
\begin{aligned}
p_A &:= (\,![B]\,.\,\mathbf{end}) + (?[C]\,\{\ell \mapsto 42\}.\,\mathbf{end}) \\
p_B &:= (\,![C]\,.\,\mathbf{end}) + (?[A]\,\{\ell \mapsto 42\}.\,\mathbf{end}) \\
p_C &:= (\,![A]\,.\,\mathbf{end}) + (?[B]\,\{\ell \mapsto 42\}.\,\mathbf{end})
\end{aligned}
$$

$\xrightarrow{p_A \text{ sends to } p_B}$	$\xrightarrow{p_B \text{ sends to } p_C}$	$\xrightarrow{p_C \text{ sends to } c_A}$
$p_A := \mathbf{end}$	$p_A := p_A$	$p_A := \mathbf{end}$
$p_B := \mathbf{end}$	$p_B := \mathbf{end}$	$p_B := p_B$
$p_C := p_C$	$p_C := \mathbf{end}$	$p_C := \mathbf{end}$

Mixtris

$$\textsc{Wp-new}\quad \frac{k = n - 1 \qquad \textsc{consistent}\ (p_0, \ldots, p_k) \qquad n > 0}{\mathsf{wp}\ \mathbf{new_chan}(n)\ \{(c_0, \ldots, c_k).\ c_0 \rightarrowtail p_0 * \cdots * c_k \rightarrowtail p_k\}}*$$

Mixtris

WP-NEW

$$\frac{k = n - 1 \qquad \textsc{consistent}\ (p_0, \ldots, p_k) \qquad n > 0}{\mathsf{wp}\ \mathtt{new_chan}(n)\ \{(c_0, \ldots, c_k).\ c_0 \rightarrowtail p_0 * \cdots * c_k \rightarrowtail p_k\}}\ *$$

WP-TRY-SEND

$$\frac{c \rightarrowtail !\,[i]\ \{P\}.\ p \qquad P}{\mathsf{wp}\ c[i].\mathtt{try_send}()\ \{b.\ \mathtt{if}\ b\ \mathtt{then}\ c \rightarrowtail p\ \mathtt{else}\ c \rightarrowtail !\,[i]\ \{P\}.\ p * P\}}\ *$$

Mixtris

WP-NEW

$$\frac{k = n - 1 \qquad \textsc{consistent}\ (p_0, \ldots, p_k) \qquad n > 0}{\mathrm{wp}\ \mathtt{new_chan}(n)\ \{(c_0, \ldots, c_k).\ c_0 \rightarrowtail p_0 * \cdots * c_k \rightarrowtail p_k\}}*$$

CHAN-SUB

$$\frac{c \rightarrowtail p_1 \qquad p_1 \sqsubseteq p_2}{c \rightarrowtail p_2}*$$

WP-TRY-SEND

$$\frac{c \rightarrowtail ![i]\ \{P\}.\ p \qquad P}{\mathrm{wp}\ c[i].\mathtt{try_send}()\ \{b.\ \mathtt{if}\ b\ \mathtt{then}\ c \rightarrowtail p\ \mathtt{else}\ c \rightarrowtail ![i]\ \{P\}.\ p * P\}}*$$

SUB-CHOICE-L

$$p + q \sqsubseteq p$$

Mixtris

WP-NEW

$$\frac{k = n - 1 \qquad \textsc{consistent}\ (p_0, \ldots, p_k) \qquad n > 0}{\mathsf{wp}\ \mathbf{new_chan}(n)\ \{(c_0, \ldots, c_k).\ c_0 \rightarrowtail p_0 * \cdots * c_k \rightarrowtail p_k\}}*$$

CHAN-SUB

$$\frac{c \rightarrowtail p_1 \qquad p_1 \sqsubseteq p_2}{c \rightarrowtail p_2}*$$

WP-TRY-SEND

$$\frac{c \rightarrowtail q \qquad q \sqsubseteq \mathbf{!}\,[i]\,\{P\}.\,p \qquad P}{\mathsf{wp}\ c[i].\mathbf{try_send}()\ \{b.\ \mathbf{if}\ b\ \mathbf{then}\ c \rightarrowtail p\ \mathbf{else}\ c \rightarrowtail q * P\}}*$$

SUB-CHOICE-L

$$p + q \sqsubseteq p$$

Mixtris

WP-NEW

$$\frac{k = n - 1 \qquad \text{CONSISTENT}\ (p_0, \ldots, p_k) \qquad n > 0}{\text{wp}\ \mathbf{new_chan}(n)\ \{(c_0, \ldots, c_k).\ c_0 \rightarrowtail p_0 * \cdots * c_k \rightarrowtail p_k\}}*$$

CHAN-SUB

$$\frac{c \rightarrowtail p_1 \qquad p_1 \sqsubseteq p_2}{c \rightarrowtail p_2}*$$

WP-TRY-SEND

$$\frac{c \rightarrowtail q \qquad q \sqsubseteq\ !\,[i]\,\{P\}.\,p \qquad P}{\text{wp}\ c[i].\mathbf{try_send}()\ \{b.\ \mathbf{if}\ b\ \mathbf{then}\ c \rightarrowtail p\ \mathbf{else}\ c \rightarrowtail q * P\}}*$$

SUB-CHOICE-L

$$p + q \sqsubseteq p$$

WP-TRY-RECV

$$\frac{c \rightarrowtail q \qquad q \sqsubseteq\ ?[i]\,\{P\}.\,p}{\text{wp}\ c[i].\mathbf{try_recv}()\ \left\{ ov.\ \begin{array}{l} \mathbf{match}\ ov\ \mathbf{with} \\ \mid \mathbf{some}\ w \Rightarrow w = () * c \rightarrowtail p * P \\ \mid \mathbf{none} \quad \Rightarrow c \rightarrowtail q \\ \mathbf{end} \end{array} \right\}}*$$

Threeway Election - Verified

Threeway election program:

$$
\begin{aligned}
&\texttt{threeway_election_example} \triangleq \\
&\quad \mathbf{let}\ \ell := \mathbf{ref}\ 42\ \mathbf{in} \\
&\quad \mathbf{let}\ (c_A, c_B, c_C) := \mathbf{new_chan}(3)\ \mathbf{in} \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_B\ C\ A\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_C\ A\ B\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}\ ; \\
&\quad \mathbf{fork}\ \{\mathbf{match}\ \texttt{send_recv}\ c_A\ B\ C\ ()\ \mathbf{with}\ \mathbf{none} \Rightarrow ();\ \mathbf{some}\ _ \Rightarrow \mathbf{free}\ \ell\ \mathbf{end}\}
\end{aligned}
$$

Threeway Election - Verified

Threeway election program:

```
threeway_election_example ≜
  let ℓ := ref 42 in
  let (c_A, c_B, c_C) := new_chan(3) in
  fork {match send_recv c_B C A () with none ⇒ (); some _ ⇒ free ℓ end};
  fork {match send_recv c_C A B () with none ⇒ (); some _ ⇒ free ℓ end};
  fork {match send_recv c_A B C () with none ⇒ (); some _ ⇒ free ℓ end}
```

Protocols:

$$c_A \rightarrowtail (![B].\mathbf{end}) + (?[C]\{\ell \mapsto 42\}.\mathbf{end})$$
$$c_B \rightarrowtail (![C].\mathbf{end}) + (?[A]\{\ell \mapsto 42\}.\mathbf{end})$$
$$c_C \rightarrowtail (![A].\mathbf{end}) + (?[B]\{\ell \mapsto 42\}.\mathbf{end})$$

Threeway Election - Verified

Threeway election program:

```
threeway_election_example ≜
  let ℓ := ref 42 in
  let (c_A, c_B, c_C) := new_chan(3) in
  fork {match send_recv c_B C A () with none ⇒ (); some _ ⇒ free ℓ end};
  fork {match send_recv c_C A B () with none ⇒ (); some _ ⇒ free ℓ end};
  fork {match send_recv c_A B C () with none ⇒ (); some _ ⇒ free ℓ end}
```

Protocols:

$$c_A \rightarrowtail (![B].\mathbf{end}) + (?[C]\{\ell \mapsto 42\}.\mathbf{end})$$
$$c_B \rightarrowtail (![C].\mathbf{end}) + (?[A]\{\ell \mapsto 42\}.\mathbf{end})$$
$$c_C \rightarrowtail (![A].\mathbf{end}) + (?[B]\{\ell \mapsto 42\}.\mathbf{end})$$

Verified functional correctness!

Concluding Remarks

More in the Paper

Full mixed choice multiparty dependent separation protocols and rules

- Dependent binders, exchanged values, value-dependent branching, recursion
- $p ::= \mathbf{!}\,[i]\,(\vec{x}:\vec{\tau})\,\langle v\rangle\{P\}.\,p \mid \mathbf{?}[i]\,(\vec{x}:\vec{\tau})\,\langle v\rangle\{P\}.\,p \mid \mathbf{end} \mid p + q \mid \mu x.p$

Committed send/recv channel primitives

- $c[i].\mathbf{send}(v)$ and $c[i].\mathbf{recv}()$
- Seamlessly compose with uncommitted send/receive

Verification of Chang and Roberts's ring leader election

- Including verification of leader agreement

Language-agnostic verification interface for MCMDSPs

- Implementation and verification of synchronisation cells

Conclusion

Mixed choice remains non-trivial to implement

- Atomic receiver commit point was crucial for synchronicity
- Proper liveness contingent on collaborative concurrency
- Unclear how to achieve in a proper distributed system

Contextual protocol consistency useful for modelling leader election

- Resources often needs to be delegated on completion
- Resources does not necessarily enter the system during the protocol

Future Work

Semantic Multiparty Mixed Choice Session Type System

- Investigate correspondences with syntactic protocol consistency

More scalable proofs of protocol consistency

- Abstraction and modularity via separation logic
- Automation via Model Checking

Deadlock freedom and liveness guarantees

- Leverage connectivity graphs for multiparty communication

Mixtris for distributed systems

- Probabilistic implementations for mixed choice

$$
\begin{aligned}
&!\,[A]\,\langle \text{“Thanks”}\rangle\{\texttt{MixtrisOverview}\}.\\
&\mu rec.\,(?[A]\,(q : \texttt{Question i})\,\langle q\rangle\{\texttt{AboutMultris}\ q\}.\\
&\qquad !\,[A]\,(a : \texttt{Answer})\,\langle a\rangle\{\texttt{Insightful}\ q\ a\}.\,rec)\\
&\qquad +\\
&\qquad !\,[C]\,\langle \text{“}\textit{Times up?}\text{”}\rangle.\,\mathbf{end}
\end{aligned}
$$

Backup Slides

Chang and Roberts's Protocol

$$
\begin{aligned}
&\texttt{cre_init_process_prot}\ (i : \mathbb{N})\ (P : \mathsf{iProp})\ (p : \mathbb{N} \to \mathsf{iProto}) : \mathsf{iProto} \triangleq \\
&\quad \mathbf{!}\,[i_l]\,\langle \mathbf{inl}\ i \rangle.\ \texttt{cre_process_prot}\ i\ P\ p\ + \\
&\quad \mathbf{?}[i_r]\,(i' : \mathbb{N})\,\langle \mathbf{inl}\ i' \rangle.\ \left\{ \begin{array}{l} \mathbf{if}\ i < i'\ \mathbf{then}\ \mathbf{!}\,[i_l]\,\langle \mathbf{inl}\ i' \rangle.\ \texttt{cre_process_prot}\ i\ P\ p \\ \mathbf{else\ if}\ i = i'\ \mathbf{then}\ \mathbf{!}\,[i_l]\,\langle \mathbf{inr}\ i \rangle.\ \texttt{cre_process_prot}\ i\ P\ p \\ \mathbf{else}\ \mathbf{!}\,[i_l]\,\langle \mathbf{inl}\ i \rangle.\ \texttt{cre_process_prot}\ i\ P\ p \end{array} \right\}
\end{aligned}
$$

Chang and Roberts's Protocol

$$
\begin{aligned}
&\texttt{cre_init_process_prot}\ (i : \mathbb{N})\ (P : \mathsf{iProp})\ (p : \mathbb{N} \to \mathsf{iProto}) : \mathsf{iProto} \triangleq \\
&\quad !\,[i_l]\,\langle \mathbf{inl}\, i\rangle.\ \texttt{cre_process_prot}\ i\ P\ p\ + \\
&\quad ?[i_r]\,(i' : \mathbb{N})\,\langle \mathbf{inl}\, i'\rangle.\ \left\{\begin{array}{l}
\mathbf{if}\ i < i'\ \mathbf{then}\ !\,[i_l]\,\langle \mathbf{inl}\, i'\rangle.\ \texttt{cre_process_prot}\ i\ P\ p \\
\mathbf{else\ if}\ i = i'\ \mathbf{then}\ !\,[i_l]\,\langle \mathbf{inr}\, i\rangle.\ \texttt{cre_process_prot}\ i\ P\ p \\
\mathbf{else}\ !\,[i_l]\,\langle \mathbf{inl}\, i\rangle.\ \texttt{cre_process_prot}\ i\ P\ p
\end{array}\right\}
\end{aligned}
$$

$$
\begin{aligned}
&\texttt{cre_process_prot}\ (i : \mathbb{N})\ (P : \mathsf{iProp})\ (p : \mathbb{N} \to \mathsf{iProto}) : \mathsf{iProto} \triangleq \mu rec. \\
&\quad \&[i_r]\left\{\begin{array}{ll}
\mathbf{inl}(i' : \mathbb{N})\langle i'\rangle & \Rightarrow \mathbf{if}\ i < i'\ \mathbf{then}\ !\,[i_l]\,\langle \mathbf{inl}\, i'\rangle.\ rec \\
 & \quad \mathbf{else\ if}\ i = i'\ \mathbf{then}\ !\,[i_l]\,\langle \mathbf{inr}\, i\rangle.\ rec \\
 & \quad \mathbf{else}\ rec \\
\mathbf{inr}(i' : \mathbb{N})\langle i'\rangle\{i = i' \twoheadrightarrow P\} & \Rightarrow \mathbf{if}\ i = i'\ \mathbf{then}\ p\ i' \\
 & \quad \mathbf{else}\ !\,[i_l]\,\langle \mathbf{inr}\, i'\rangle.\ p\ i'
\end{array}\right\}
\end{aligned}
$$

Implementation - Synchronisation Cells

```
new_sync () := ref none

wait c := match !c with
          | none  ⇒ ()
          | some _ ⇒ wait c
          end

sync_put c v := c ← some v;
                wait c.

sync_get c :=
  match Xchg c none with
  | none  ⇒ sync_get c
  | some v ⇒ v
  end

sync_try_put c v :=
  c ← some v;
  match Xchg c none with
  | none  ⇒ true
  | some _ ⇒ false
  end

sync_try_get c := Xchg c none
```

Implementation - Channels

Array of synchronisation cells

$$\mathbf{new_chan}(n) := \mathbf{let}\ m := \texttt{new_matrix}\ n\ n\ (\texttt{new_sync}())\ \mathbf{in}\ ((m,0),\ldots,(m,n-1))$$

$$c[j].\mathbf{send}(v) := \mathbf{let}\ (m,i) := c\ \mathbf{in}\ \texttt{sync_put}\ m_{i,j}\ v$$

$$c[j].\mathbf{recv}() := \mathbf{let}\ (m,i) := c\ \mathbf{in}\ \texttt{sync_get}\ m_{j,i}$$

$$c[j].\mathbf{try_send}(v) := \mathbf{let}\ (m,i) := c\ \mathbf{in}\ \texttt{sync_try_put}\ m_{i,j}\ v$$

$$c[j].\mathbf{try_recv}() := \mathbf{let}\ (m,i) := c\ \mathbf{in}\ \texttt{sync_try_get}\ m_{j,i}$$

Mixtris Ghost Theory

We defined the our protocols via Iris's recursive domain equation solver and proved language-generic ghost theory rules based on Iris's ghost state machinery

PROTO-ALLOC

$$\frac{\textsc{consistent}\ \vec{p}}{\Rrightarrow \exists \chi.\ \mathsf{prot_ctx}\ \chi\ |\vec{p}| * \mathop{\ast}_{i \mapsto p \in \vec{p}} \mathsf{prot_own}\ \chi\ i\ p}\ *$$

PROTO-LE

$$\frac{\mathsf{prot_own}\ \chi\ i\ p_1 \qquad p_1 \sqsubseteq p_2}{\mathsf{prot_own}\ \chi\ i\ p_2}\ *$$

PROTO-STEP

$$\frac{\mathsf{prot_ctx}\ \chi\ n \qquad P_1 \qquad \mathsf{prot_own}\ \chi\ i\ (!\,[j]\,\{P_1\}.\,p_1) \qquad \mathsf{prot_own}\ \chi\ j\ (?[i]\,\{P_2\}.\,p_2)}{\Rrightarrow \triangleright \mathsf{prot_ctx}\ \chi\ n * \mathsf{prot_own}\ \chi\ i\ p_1 * \mathsf{prot_own}\ \chi\ j\ p_2\ * P_2}\ *$$

One can then define language-specific $c \rightarrowtail p$ and prove Hoare triple rules (such as WP-TRY-SEND, WP-TRY-RECV, and WP-NEW) for an implementation using the ghost theory